

Gradient Normalization Enables Communication-Efficient Distributed Learning Under Initialization Data Heterogeneity

Tao Sun [✉], Baihao Wu, Xinwang Liu [✉], *Senior Member, IEEE*, and Kun Yuan [✉], *Member, IEEE*

Abstract—Communication-efficient distributed learning has achieved remarkable progress in training large-scale deep neural networks across numerous clients. However, in heterogeneous environments—where local data distributions vary significantly—the empirical performance of many such algorithms degrades sharply. Moreover, existing theoretical analyses often depend on overly restrictive assumptions, such as bounded data heterogeneity, which may not be valid even for a single client. In this work, we introduce a general gradient normalization strategy that can be seamlessly integrated into a wide range of distributed learning algorithms, including compressed distributed stochastic gradient descent, federated averaging, and asynchronous variants. Our theoretical analysis demonstrates that this normalization technique effectively mitigates the negative impact of data heterogeneity, allowing these algorithms to achieve linear speedup rates with only requiring the boundedness of initialization data heterogeneity. Extensive numerical experiments further confirm the practical effectiveness and theoretical guarantees of our approach.

Index Terms—Asynchronous learning, communication compression, data heterogeneity, distributed learning, federated learning, gradient normalization, momentum.

I. INTRODUCTION

THIS paper considers a distributed machine learning scenario in which n agents collaborate to solve the following stochastic optimization problem

$$\min_{\mathbf{w} \in \mathbb{R}^d} f(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{w}), \quad (1)$$

where $f_i(\mathbf{w}) := \mathbb{E}_{\xi_i \sim \mathcal{D}_i} f(\mathbf{w}; \xi_i)$. The random variable ξ_i represents the local data sample kept in agent i , following a local distribution $\mathcal{D}_i$. Since $\mathcal{D}_i$ is typically unknown in advance, each agent i can only access its stochastic gradient $\nabla f(\mathbf{w}; \xi_i)$ per

iteration, rather than the true local gradient $\nabla f_i(\mathbf{w})$. Problem (1) is referred to as *homogeneous distributed learning* if all local distributions $(\mathcal{D}_i)_{1 \leq i \leq n}$ are identical; otherwise it is referred to as *heterogeneous distributed learning*.

Distributed stochastic gradient descent (SGD) [1], [2], [3] stands as a straightforward yet powerful algorithm for solving problem (1). During iteration t of distributed SGD, the central server broadcasts the current model parameter $\mathbf{w}^t$ to client i , which then computes the local stochastic gradient. Subsequently, the server aggregates all gradients from the clients and performs the following update

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \frac{\gamma}{n} \sum_{i=1}^n \mathbf{g}^t(i), \quad \text{where } \mathbf{g}^t(i) = \nabla f(\mathbf{w}^t; \xi_i^t) \quad (\text{distributed SGD})$$

with γ denoting the learning rate. It is worth noting that distributed SGD corresponds to mini-batch SGD [4], which implies that the performance of distributed SGD remains consistent for both heterogeneous and homogeneous distributed learning scenarios. In other words, distributed SGD can converge under the same assumptions and with the same convergence rate in both scenarios, indicating that it tackles heterogeneous distributed learning as easily as the homogeneous counterpart.

A. Communication-Efficient Distributed SGD

While vanilla distributed SGD excels in theoretical convergence rate, it faces a notable communication overhead due to the need for global synchronization across all clients and the transmission of full-dimensional gradients in every iteration. To mitigate this issue, several well-known communication-efficient algorithms are proposed in the literature:

- *Compressed distributed SGD (CD-SGD)*: CD-SGD [5], [6], [7] aims to reduce the volume of information exchanged between clients and the server by transmitting compressed gradients or models. This compression is achieved through either quantizing each entry to a smaller set of discrete values or dropping a certain amount of entries to obtain a sparse vector.
- *Federated averaging (FedAvg)*: FedAvg [8], [9], [10] aims to reduce communication frequency by employing lazy communication that avoids unnecessary information exchange between clients and the central server. This strategy allows each client to either conduct a fixed number of

Received 8 May 2025; revised 4 February 2026; accepted 25 April 2026. Date of publication 1 May 2026; date of current version 6 August 2026. The work of Tao Sun was supported in part by the National Natural Science Foundation of China under Grant 62522610 and Grant 62376278, and in part by NUDT Foundational Research Funding (JS25-02). Recommended for acceptance by T. Yang. (Tao Sun and Baihao Wu contributed equally to this work.) (Corresponding authors: Tao Sun; Xinwang Liu.)

Tao Sun and Xinwang Liu are with the College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China (e-mail: suntao.saltfish@outlook.com; xinwangliu@nudt.edu.cn).

Baihao Wu and Kun Yuan are with the Peking University, Beijing 100871, China (e-mail: baihaowu@pku.edu.cn; kunyuan@pku.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPAMI.2026.3689520>, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2026.3689520

local updates before sending messages or adaptively skip communications when necessary.

- *Asynchronous distributed SGD (AD-SGD)*: AD-SGD [11], [12], [13] aims to minimize idle time, especially in the presence of straggler clients or clients with varying computation speeds. In asynchronous SGD, each client initiates the next computation immediately after finishing the computation of its own gradient, without waiting for any other clients.

These algorithms have demonstrated great success in homogeneous distributed learning. However, their performances deteriorate both empirically and theoretically when each local data distribution $\mathcal{D}_i$ differs significantly from each other. In practical implementations, substantial data heterogeneity can significantly slow down the convergence of the aforementioned algorithms [14], [15], [16]. In theory, the analyses of these algorithms rely heavily on an additional but fundamental assumption concerning bounded data heterogeneity. A general formulation of the bounded data heterogeneity is proposed in [17], [18]:

$$\sup_{\mathbf{w} \in \mathbb{R}^d} \left\{ \sum_{i=1}^n \frac{1}{n} \|\nabla f(\mathbf{w}) - \nabla f_i(\mathbf{w})\|^2 \right\} \leq \zeta^2 + D \|\nabla f(\mathbf{w})\|^2, \quad (2)$$

where $\zeta \geq 0$ quantifies the extent of data heterogeneity, and $D \geq 0$. When $\zeta = 0$ and $D = 0$, it implies that $\nabla f_i(\mathbf{w}) = \nabla f(\mathbf{w})$ for all $i \in \{1, \dots, n\}$, which occurs when all local data distributions $\mathcal{D}_i$ are identical. Note that the aforementioned algorithms do not rely on this assumption in homogeneous distributed learning. However, without the assumption stated in (2), most of the convergence analyses for these algorithms found in existing literature become invalid under heterogeneous distributed learning.

It is important to note that assumption (2) is highly restrictive and may even be unrealistic or redundant. To illustrate this, let us consider a scenario where $n = 2$, $f_2(\mathbf{w}) \equiv -f_1(\mathbf{w})$. In this case, the distributed learning reduces to a single-server algorithm that minimizes $f(\mathbf{w}) \equiv 0$. Consequently, the assumption of bounded data heterogeneity (2) becomes the bounded gradient assumption, which states as $\sup_{\mathbf{w} \in \mathbb{R}^d} \|\nabla f_1(\mathbf{w})\| \leq \zeta^2$ that is not commonly utilized in stochastic algorithms. In fact, it has been observed that the gradients can be divergent in deep learning tasks [19]. On the other hand, the objective $f(\mathbf{w}) \equiv 0$ is considered trivial; any assumptions made about the function f_1 are redundant.

B. Motivating Open Question

While vanilla distributed SGD maintains similar performance regardless of data heterogeneity, communication-efficient algorithms like CD-SGD, FedAvg, and AD-SGD encounter greater challenges with heterogeneous data compared to homogeneous data. Specifically, these communication-efficient methods necessitate more restrictive assumptions on data heterogeneity and demonstrate substantially slower convergence rates resulting from heterogeneous data effects.

Significant research efforts have focused on improving communication-efficient SGD algorithms, with the goal of making them achieve similar theoretical and empirical performance

regardless of data heterogeneity. For CD-SGD, techniques like error feedback [5], [20], [21], [22] and multi-step compression strategies [23], [24] have proven critical for enhancing performance under heterogeneous data. Regarding FedAvg, adoption of methods including control variables [17], diminishing local learning rates [25], momentum acceleration [26], and extremely high-precision local problem solving [27], [28] have helped address heterogeneity issues.

While these techniques have demonstrated effectiveness in their respective algorithms, they lack the flexibility to be applied to different scenarios. For instance, reducing local learning rates benefits FedAvg but cannot be applied to CD-SGD and AD-SGD. Furthermore, none of the aforementioned techniques have been successfully adapted to AD-SGD to our knowledge, which highlights the requirement for further research on enhancing the performance of AD-SGD under data heterogeneity. Currently, a major challenge in heterogeneous distributed learning is the need to study each communication-efficient algorithm independently and develop heterogeneity-robust techniques that are specifically tailored to that algorithm. This process clearly demands significant manual efforts and expertise, which motivates the following open question:

Is there a general technique that can make communication-efficient heterogeneous distributed learning almost as easy as its homogeneous counterpart?

In the question above, we use “general” to imply a unified technique that can be easily applied to various distributed learning approaches with minor or even no adjustments in algorithmic structures. By “communication-efficient heterogeneous distributed learning”, we specifically refer to distributed learning with compressed, federated, or asynchronous communication under data heterogeneity. When we say “almost as easy as”, we mean that the technique enables a certain algorithm to converge under almost same assumptions (i.e., without heterogeneity bound (2) but with a simple initialization condition) and with comparable convergence rate, computational cost, and memory requirement in both homogeneous and heterogeneous setting.

C. Main Results and Contributions

This paper provides a solution to the open question posed above. Specifically, we have developed a general technique, termed *gradient normalization with momentum*, which can be readily applied to distributed learning incorporating compressed, federated, and asynchronous communications, under a much weaker condition, namely the initialization data heterogeneity defined as

$$\zeta_0^2 := \sum_{i=1}^n \frac{1}{n} \|\nabla f(\mathbf{w}^0) - \nabla f_i(\mathbf{w}^0)\|^2 < +\infty. \quad (3)$$

This condition is imposed only at the initialization point $\mathbf{w}^0$, and is therefore strictly weaker than the standard uniform data heterogeneity measure; in particular, one typically has $\zeta_0 \ll \zeta$. This improves upon various existing algorithms to achieve comparable convergence performance under the same assumptions, for both homogeneous and heterogeneous settings. Our main results are summarized below:

- The recently proposed CD-SGD algorithm with multi-step compression [23], [24] nearly achieves optimal convergence rates for distributed learning with compressed communications. However, its convergence relies on the data heterogeneity bound in (2). We apply gradient normalization to CD-SGD with multi-step compression and develop a novel algorithm guaranteed to converge without needing (2), while still achieving provable linear speedup concerning the number of clients.
- FedAvg has emerged as a prevalent algorithm for federated learning, demonstrating great success in various applications. However, its convergence performance suffers notably from data heterogeneity. By integrating gradient normalization to FedAvg, we have built a new federated learning algorithm with a state-of-the-art convergence rate, without requiring any bounded data heterogeneity assumptions.
- AD-SGD proves highly effective in the presence of straggler clients but faces theoretical and empirical challenges arising from heterogeneous data distributions across different nodes. We leverage gradient normalization to augment AD-SGD, achieving the first asynchronous algorithm that converges without relying on assumption (2). Furthermore, we establish a convergence rate matching that of AD-SGD under bounded delay, surpassing the recent rate established by [12].

We would like to emphasize two key points of the contributions: 1) Our results in this paper have successfully eliminated the bounded heterogeneous data assumption in the three distributed algorithms and demonstrated convergence rates comparable to that under the bounded heterogeneous data assumption. These results underscore the effectiveness of the “universal” technique, gradient normalization, in eliminating the bounded heterogeneous data assumption in distributed settings, with potential applications in other distributed environments. 2) We do not claim that gradient normalization with momentum is uniformly the strongest approach in every distributed setting. Depending on the structure or information available in a specific setting, it is total possible for researchers to design more efficient algorithms tailored to that context, which may not be addressed by a “universal” method. But this fact does not diminish our contribution because our focus is on developing a general method that can be applied across various scenarios, emphasizing the versatility and applicability of gradient normalization in distributed learning settings. Nevertheless, this positioning statement should not be interpreted as “not achieving the best-known rate”: our results match the minimax-optimal dominant complexity under our stationary metric and the standard stochastic first-order oracle.

Notation: This paper uses $\mathbb{E}[\cdot]$ to denote the expectation with respect to the underlying probability space and $\|\cdot\|$ to denote the L_2 -norm of a vector. The minimum value of the function f is denoted as $\min f$. Given two sequences $\{a_t\}$ and $\{b_t\}$, we write $a_t = \mathcal{O}(b_t)$ if there exists a positive constant $0 < C < +\infty$ such that $a_t \leq Cb_t$. We write $a_t = \Omega(b_t)$ if there exist constants $0 < c_1 \leq c_2 < \infty$ and t_0 such that $c_1 b_t \leq a_t \leq c_2 b_t$ for all $t \geq t_0$.

Throughout the paper, we adopt the convention $\frac{0}{0} = 0$ to ensure that the gradient normalization at $\mathbf{0}$ is valid.

D. More Related Works

Distributed learning with compressed communication: Communication compression, through quantization or sparsification approaches, is a widely used method for enabling communication-efficient distributed training. Quantization approximates stochastic gradients using low-bit representations with unbiased [29], [30], [31] or biased techniques [32]. An extreme quantization approach called Sign-SGD uses only the sign information but demonstrates impressive empirical performance [5], [7], [33]. Sparsification involves selectively dropping elements from the communicated vectors, either randomly to maintain unbiased stochastic gradients [34], or by keeping only the top elements with the largest absolute values (Top- K) [35], [36]. Such biased compression methods trade-off information completeness for communication efficiency. Recent work has thus focused on compensating for missing information via error feedback to eliminate compression-induced errors [20], [21], [37]. Most recently, [23] derived a lower bound for compressed SGD and proposed a nearly-optimal algorithm.

Distributed learning with federated communication: Early works on FedAvg primarily focused on homogeneous data, referred to as local SGD or parallel SGD [9], [38], [39], [40], [41]. These approaches assume that clients perform multiple local gradient computations to reduce communication overhead in distributed systems. In the convex case, [42], [43], [43], [44], [45] investigate the upper convergence bound of FedAvg with multiple averaging rounds. The analysis of FedAvg on both homogeneous and heterogeneous nonconvex scenarios is explored in [40], [46], [47], [48]. The results demonstrate that FedAvg significantly enhances communication efficiency and mitigates the influence of bounded variance. Within the federated optimization framework, several variations of FedAvg have been developed for different settings or objectives [17], [49], [50], [51], [52]. A recent work [53] relaxes the bounded data heterogeneity assumption (2) to a condition in terms of objective function values.

Distributed learning with asynchronous communication: Asynchrony is commonly used to accelerate parallel and distributed training by reducing client waiting time [54]. Analyzing the convergence of asynchronous SGD involves studying the delay in each iteration. Early works assumed that the delay is uniformly bounded by a constant [11], [55], [56]. More recently, [12] presented sharper convergence results for asynchronous SGD in distributed optimization. Aside from uniformly bounded delays, there is also research on unbounded delays [57], [58], [59]. In [13], the authors prove an interesting result: asynchronous SGD performs better than mini-batch SGD under arbitrary delays when prior information about the delays in each iteration is available for step size selection. The paper [60] presents a new protocol that expands the classical oracle framework to analyze minimax complexities for parallel optimization methods using an unbiased stochastic gradient

oracle with bounded variance. In a fixed computation model, each worker has a fixed but worker-dependent time to compute stochastic gradients. The study establishes lower bounds and devises optimal algorithms, offering novel insights for asynchronous optimization methods.

Gradient normalization: Rather than using the (stochastic) gradient, gradient normalization involves projecting the gradient onto a sphere, resulting in different convergence performances and analyses. Interestingly, the biased scheme benefits SGD in quasi-convex settings [61]. In [62], the authors demonstrate that gradient normalization improves the ability to escape saddle points. Additionally, if extra smooth assumptions are met, gradient normalization can even accelerate convergence rate [63]. However, under the same assumptions as SGD, gradient normalization fails to converge without a fixed momentum [64]. Recently, [65] have applied gradient normalization techniques to SGD with Markov noise and demonstrated the advantages of the algorithm in scenarios with weak smoothness.

II. GRADIENT NORMALIZATION WITH MOMENTUM

A. Assumptions

We make the following assumptions throughout the paper, which are commonly utilized in stochastic optimization.

Assumption 1 (Smoothness): We assume each local function $f_i(\mathbf{w})$ is L -smooth, i.e., there exists a constant $L > 0$ such that $\forall i \in \{1, 2, \dots, n\} \|\nabla f_i(\mathbf{x}) - \nabla f_i(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$, $\forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^d$.

Assumption 2 (Stochastic gradient). We assume each stochastic gradient $\nabla f(\mathbf{w}; \xi_i)$ satisfies $\mathbb{E}_{\xi_i \sim \mathcal{D}_i}[\nabla f(\mathbf{w}; \xi_i)] = \nabla f_i(\mathbf{w})$, $\mathbb{E}_{\xi_i \sim \mathcal{D}_i}[\|\nabla f(\mathbf{w}; \xi_i) - \nabla f_i(\mathbf{w})\|^2] \leq \sigma^2$, for any $\mathbf{w} \in \mathbb{R}^d$ and $i \in \{1, 2, \dots, n\}$. The constant $\sigma^2 > 0$ denotes the variance of stochastic gradient. Moreover, we assume each sampled ξ_i is independent of each other for any $i \in \{1, 2, \dots, n\}$.

B. Why is Bounded Data Heterogeneity Required by Prior Works?

As we discussed in Section I, existing well-known algorithms such as CD-SGD with multi-step compression, FedAvg, and AD-SGD all require bounded data heterogeneity assumption (2) to converge. We provide an overview of the analysis for the previous distributed algorithm under the assumption (2). For simplicity, let us denote $\mathbf{g}^t(i)$ and $\hat{\mathbf{g}}^t(i)$ as the stochastic gradient and transmitted one of node i in the t -th iteration, respectively. Their averages are denoted by $\mathbf{g}^t$ and $\hat{\mathbf{g}}^t$, respectively. The update step of the distributed algorithms is expressed as $\mathbf{w}^{t+1} = \mathbf{w}^t - \gamma \frac{\sum_{i=1}^n \hat{\mathbf{g}}^t(i)}{n}$. With gradient Lipschitz property, it follows¹

$$\begin{aligned} \mathbb{E}f(\mathbf{w}^{t+1}) - \mathbb{E}f(\mathbf{w}^t) &\leq -\left(\frac{\gamma}{2} - L\gamma^2\right) \mathbb{E}\|\nabla f(\mathbf{w}^t)\|^2 \\ &\quad + \frac{L\gamma^2\sigma^2}{n} + \left(\frac{\gamma}{2} + L\gamma^2\right) \mathbb{E}\|\mathbf{g}^t - \hat{\mathbf{g}}^t\|^2. \end{aligned}$$

¹Please refer to details in the Appendix.

The protocol of the communications in the distributed algorithms can result in

$$\mathbb{E}\|\mathbf{g}^t - \hat{\mathbf{g}}^t\|^2 = \mathcal{O}\left(\sum_{i=1}^n \mathbb{E}\|\nabla f_i(\mathbf{w}^t)\|^2/n + C\right)$$

for some $C \geq 0$. For example in the compressed distributed algorithms, $\|\mathbf{g}^t(i) - \hat{\mathbf{g}}^t(i)\|^2 \leq c\|\mathbf{g}^t(i)\|^2$ with $c > 0$. We can get $\mathbb{E}\|\mathbf{g}^t - \hat{\mathbf{g}}^t\|^2 \leq c\sum_{i=1}^n \mathbb{E}\|\nabla f_i(\mathbf{w}^t)\|^2/n + c\sigma^2$. In the analyses above, assumption (2) provides an upper bound for $\sum_{i=1}^n \mathbb{E}\|\nabla f_i(\mathbf{w}^t)\|^2/n$, connecting the local gradients and the global gradient, as follows

$$\sum_{i=1}^n \mathbb{E}\|\nabla f_i(\mathbf{w}^t)\|^2/n \leq \|\nabla f(\mathbf{w}^t)\|^2 + \zeta^2.$$

Without data heterogeneity bound (2), these communication-efficient distributed algorithms cannot derive a bound for $\sum_{i=1}^n \mathbb{E}\|\nabla f_i(\mathbf{w})\|^2/n$, which fundamentally hinders their analyses.

C. Normalized Update

Indeed, the convergence analyses of these communication-efficient stochastic algorithms rely only on the bound of $\sum_{i=1}^n \mathbb{E}\|\nabla f_i(\mathbf{w})\|^2/n$ for the algorithm trajectories $\{\mathbf{w}^t\}_{t \geq 0}$ rather than for any $\mathbf{w} \in \mathbb{R}^d$, which brings a chance to develop algorithms without needing assumption (2). To see this, let us introduce normalization to the variable update of one certain communication-efficient distributed algorithm

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \gamma \frac{\mathbf{m}^t}{\|\mathbf{m}^t\|}, \quad (4)$$

where $\mathbf{m}^t$ is a gradient estimate that incorporates some (or all) historical stochastic gradients from the iteration trajectories. It follows from (4) that

$$\begin{aligned} \|\nabla f_i(\mathbf{w}^{t+1})\| &\leq \|\nabla f_i(\mathbf{w}^{t+1}) - \nabla f_i(\mathbf{w}^t)\| + \|\nabla f_i(\mathbf{w}^t)\| \\ &= \|\nabla f_i(\mathbf{w}^t)\| + L\|\mathbf{w}^{t+1} - \mathbf{w}^t\| = \|\nabla f_i(\mathbf{w}^t)\| + \gamma L \quad (5) \end{aligned}$$

for $t \leq T$, where T is the number of iterations. Apparently, the last equality holds due to the introduction of the normalization. In this way, we obtain the bound $\|\nabla f_i(\mathbf{w}^t)\| = \mathcal{O}(\gamma T + \|\nabla f_i(\mathbf{w}^0)\|)$, which further implies $\sum_{i=1}^n \mathbb{E}\|\nabla f_i(\mathbf{w}^t)\|^2/n = \mathcal{O}(\gamma^2 T^2 + \sum_{i=1}^n \mathbb{E}\|\nabla f_i(\mathbf{w}^0)\|^2/n)$. By choosing a proper learning rate γ , we can easily bound on $\sum_{i=1}^n \mathbb{E}\|\nabla f_i(\mathbf{w}^t)\|^2/n$ by utilizing a term that is dependent on the value of T and $\sum_{i=1}^n \mathbb{E}\|\nabla f_i(\mathbf{w}^0)\|^2/n$.

D. Momentum Ensures the Convergence With Gradient Normalization

In the normalized update (4), any choice of the gradient estimate $\mathbf{m}^t$ will result in the desired gradient bound (5). However, the variance of the estimated stochastic gradient will significantly influence the convergence performance of the normalized stochastic algorithm. It has been observed in [61], [66], [67] that the normalized stochastic algorithm requires the variance of the gradient oracle to be very small in order to achieve any convergence guarantees.

Inspired by recent work [64], this paper utilizes gradient normalization with momentum

$$\mathbf{m}^t = \theta \mathbf{m}^{t-1} + (1 - \theta) \nabla f(\mathbf{w}^t; \xi^t), \quad (6)$$

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \gamma \frac{\mathbf{m}^t}{\|\mathbf{m}^t\|}, \quad (7)$$

and adapts it to various communication-efficient distributed learning scenarios. In the above update, $\nabla f(\mathbf{w}^t; \xi^t)$ denotes the stochastic estimate of $\nabla f(\mathbf{w}^t)$ and $\theta \in (0, 1)$ denotes the momentum coefficient. Intuitively, momentum serves as a surrogate for large-batch stochastic gradient, thus resulting in smaller gradient variance. To see this, the momentum update (6) can be rewritten as

$$\mathbf{m}^t = \sum_{k=1}^t \alpha_k \nabla f(\mathbf{w}^k; \xi^k),$$

where $\alpha_k = \theta^{t-k}(1 - \theta)$. As the iteration count t increases, the sequence $(\mathbf{w}^k)_{k \geq 0}$ tends to stabilize, resulting in the coefficients α_k diminishing for small k values. Consequently, a large positive integer $M < t$ can be identified, such that the approximation $\mathbf{m}^t \approx \sum_{k=1}^M \alpha_k \nabla f(\mathbf{w}^k; \xi^k)$ holds. This indicates that $\mathbf{m}^t$ encapsulates a large batch of stochastic gradients, leading to a smaller gradient variance. This insight also bears significance for momentum scheduling, suggesting that to achieve a larger M , one must avoid excessively reducing the momentum parameter θ , as this causes rapid decay of the coefficients $(\alpha_k)_{1 \leq k \leq t}$.

III. CD-SGD WITH GRADIENT NORMALIZATION

Starting from this section, we will apply gradient normalization to different communication-efficient distributed learning scenarios. Our aim is to theoretically validate how this technique facilitates the convergence of several popular algorithms. Specifically, we show that with gradient normalization, these algorithms can converge under the same assumptions (i.e., without heterogeneity bound (2)) and achieve comparable convergence rates, computational costs, and memory requirements in both homogeneous and heterogeneous settings.

A. Contractive Compression

The contractive compressor is widely employed in communication compression. Although it may introduce bias, it typically results in significantly less variance in compression. The definition of a contractive compressor is presented as follows.

Definition 1 (Contractive Compressor): Let $\mathcal{C} : \mathbb{R}^d \mapsto \mathbb{R}^d$ be the (possibly random) compression operator. We define $\mathcal{C}$ as a δ -contractive compressor if there exists a constant $\delta \in (0, 1]$ such that

$$\mathbb{E}_{\mathcal{C}} \|\mathcal{C}(\mathbf{x}) - \mathbf{x}\|^2 \leq (1 - \delta) \|\mathbf{x}\|^2, \quad \forall \mathbf{x} \in \mathbb{R}^d. \quad (8)$$

It is noteworthy that when $\delta = 1$, the contractive compressor reduces to the identity operator, implying no compression is applied to $\mathbf{x}$. Another commonly used contractive compressor is the Top- K sparsification [36], where only the largest K elements are selected to be transmitted, while the others remain uncommunicated.

B. Distributed SGD With Multi-Step Compression

In distributed learning, [23] and [24] propose a distributed SGD algorithm with multi-step compression that nearly attains the optimal convergence rate. This algorithm is built upon a multi-step compression (MSC) operator which can significantly reduce the compression error with an increase in the number of compressions per algorithmic iteration. The definition of the MSC operator is presented as follows.

Definition 2 (Multi-step Compression): Given a compression operator $\mathcal{C} : \mathbb{R}^d \mapsto \mathbb{R}^d$ and a positive integer $u \in \mathbb{Z}^+$, the multi-step compression (MSC) operator with u rounds and compressor $\mathcal{C}$, $\text{MSC}_{\mathcal{C}}^u : \mathbb{R}^d \rightarrow \mathbb{R}^d$, is defined as

$$\text{MSC}_{\mathcal{C}}^u(\mathbf{x}) := \sum_{k=0}^{u-1} \mathbf{c}^k \quad (9)$$

where $\mathbf{v}^0 = \mathbf{0}$, and $\mathbf{c}^k = \mathcal{C}(\mathbf{x} - \mathbf{v}^k)$, $\mathbf{v}^{k+1} = \mathbf{v}^k + \mathbf{c}^k$ as $k \in \{0, 1, \dots, u-1\}$.

The most favorable property of the MSC operator is established as follows: the compression error induced by the MSC operator diminishes exponentially to 0 as u increases.

Lemma 1 ([23]): For a δ -contractive compressor $\mathcal{C} : \mathbb{R}^d \mapsto \mathbb{R}^d$, it holds for any $u \geq 1$ that

$$\mathbb{E}_{\mathcal{C}} \|\text{MSC}_{\mathcal{C}}^u(\mathbf{x}) - \mathbf{x}\|^2 \leq (1 - \delta)^u \|\mathbf{x}\|^2, \quad \forall \mathbf{x} \in \mathbb{R}^d.$$

With the MSC operator in Definition 2, the non-convex distributed SGD algorithm with multi-step compression [24] is listed in Algorithm 4 in Appendix A, online available. While distributed SGD with multi-step compression can nearly attain the optimal convergence rate in the non-convex scenario, its analysis relies on the restrictive assumption (2) in the heterogeneous setting.

C. The Gradient Normalization Algorithm for CD-SGD

To remove the reliance on the restrictive assumption (2) and improve the convergence of distributed SGD with multi-step compression in heterogeneous scenarios, we propose a novel distributed normalized SGD algorithm outlined in Algorithm 1. This new algorithm distinguishes itself from Algorithm 4 in the following aspects.

- It eliminates the necessity for sampling large-batch stochastic gradients.
- It incorporates and compresses momentum per iteration.
- It employs gradient normalization, in contrast to the conventional gradient descent, for parameter updates after aggregating information from all clients.

Significantly, it is observed that the aforementioned modification only marginally increases storage costs, associated with the introduction of momentum, and computational costs, attributed to the calculation of the momentum norm. Importantly, this adjustment does not introduce additional communication in each iteration.

D. Convergence Analysis

The following theorem establishes the convergence property of distributed normalized SGD with multi-step compression.

Algorithm 1: Distributed Normalized SGD With MSC.

Require: hyperparameters $\gamma > 0$, $0 \leq \theta < 1$, $u > 0$ and compression operator $\mathcal{C}$
Initialization: $\mathbf{w}^0$, $\mathbf{m}^0 = \mathbf{0}$
for iteration $t = 1, 2, \dots, T/u$
 communicate the parameter to all clients
 one client $i \in \{1, 2, \dots, n\}$ **in parallel do**
 step 1: Client i samples $(\xi_i^{t,k})_{1 \leq k \leq u} \sim \mathcal{D}_i$ to get
 $\mathbf{g}^t(i) = \frac{\sum_{k=1}^u \nabla f(\mathbf{w}^t; \xi_i^{t,k})}{u}$
 step 2: Client i updates
 $\mathbf{m}^t(i) = \theta \mathbf{m}^{t-1}(i) + (1 - \theta) \mathbf{g}^t(i)$
 step 3: Client i sends $\bar{\mathbf{m}}^t(i) = \text{MSC}_C^u(\mathbf{m}^t(i))$ to Server
 end on client
 step 4: Server runs $\mathbf{w}^{t+1} = \mathbf{w}^t - \gamma \frac{\sum_{i=1}^n \bar{\mathbf{m}}^t(i)}{\|\sum_{i=1}^n \bar{\mathbf{m}}^t(i)\|}$
end for

Theorem 1: Let $(\mathbf{w}^t)_{t \geq 0}$ be generated by Algorithm 1. Suppose Assumptions 1-2 hold and $\mathcal{C}$ be the contractive compressor with $\delta \in (0, 1]$. If $1 - \theta = \min \left\{ \frac{\sqrt{nu}}{\sigma\sqrt{T}}, 1 \right\}$, $\gamma = \frac{\sqrt{1-\theta}}{L\sqrt{T}}$, and $u \geq \lceil 2 \ln(T) \rceil / \lceil \ln \frac{1}{1-\delta} \rceil$, then it holds that

$$\begin{aligned} \frac{1}{R} \sum_{t=1}^R \mathbb{E} \|\nabla f(\mathbf{w}^t)\| &\leq \frac{L\sqrt{\sigma}(f(\mathbf{w}^0) - \min f) + 9\sqrt{\sigma}}{n^{1/4}T^{1/4}} \\ &+ \frac{L\sqrt{u}(f(\mathbf{w}^0) - \min f) + 2\|\nabla f(\mathbf{w}^0)\|\sqrt{u}\sigma + 9\sqrt{u}}{\sqrt{T}} \\ &+ \frac{4u\zeta_0 + 6u\|\nabla f(\mathbf{w}^0)\| + 4u\sigma}{T}, \end{aligned}$$

where ζ_0 is a constant defined by (3) and $R = \frac{T}{u}$.

Linear speedup without assuming bounded data heterogeneity: According to the above theorem, the proposed algorithm converges at a rate of $\mathcal{O}\left(\frac{\sqrt{\sigma}}{n^{1/4}T^{1/4}}\right)$ when T becomes sufficiently large. This implies that to achieve an ϵ -accurate solution, the algorithm requires $\mathcal{O}(\sigma^2/(n\epsilon^4))$ iterations, exhibiting a linear reduction with respect to the produce of the number of clients n . Notably, this linear speedup rate attains the lower bound in non-convex stochastic learning derived by [68], indicating that it cannot be further improved beyond the dominant rate of $\mathcal{O}\left(\frac{1}{n^{1/4}T^{1/4}}\right)$. More significantly, this linear speedup rate is achieved without relying on any bounded data heterogeneity assumptions (e.g. condition (2)) that are prevalent in existing algorithms with communication compression.

Comparison with classical algorithms: In the homogeneous distributed learning, various algorithms employing contractive compression can asymptotically attain the linear speedup rate of $\mathcal{O}\left(\frac{\sqrt{\sigma}}{n^{1/4}T^{1/4}}\right)$ without imposing restrictive assumptions (see e.g., [23]). However, achieving linear speedup for heterogeneous distributed learning has proven much more challenging without strong conditions. For instance, methods like [30], [36], [69] assume bounded gradients, i.e., $\sup_{\mathbf{w} \in \mathbb{R}^d, i \in \{1, 2, \dots, n\}} \{\mathbb{E}_{\xi_i \sim \mathcal{D}_i} [\|\nabla f(\mathbf{w}; \xi_i)\|]\} \leq G$, while [23],

[24] presume bounded data heterogeneity (i.e., condition 2) to derive the linear speedup result.

Comparison with EF21 and EF21-SGDM: An enhanced error feedback approach was introduced in [21] called EF21 to eliminate the uniformly bounded gradient assumptions, but only achieves $\mathcal{O}\left(\frac{\sqrt{\sigma}}{T^{1/4}}\right)$ convergence, which lacks the scalability to client size n . Later, the authors of [22] proposed EF21-SGDM, which applies Polyak's momentum to the EF21 mechanism, achieving a nonconvex linear speedup rate without relying on bounded data heterogeneity. As long as $\sigma > 0$, EF21-SGDM chooses the learning rate and momentum to scale as $\Omega\left(\frac{\sqrt{n}}{\sigma\sqrt{T}}\right)$ and $1 - \Omega\left(\frac{\sqrt{n}}{\sigma\sqrt{T}}\right)$, respectively. In our method, the momentum parameter is set to $1 - \Omega\left(\frac{\sqrt{\ln T} \sqrt{n}}{\sqrt{\ln \frac{1}{1-\delta}} \sigma\sqrt{T}}\right)$, which matches the same n, T, σ scaling as EF21-SGDM up to mild logarithmic factors in T and the compression constant δ . In contrast, our learning rate is smaller, formulated as $\Omega\left(\frac{(\ln T)^{\frac{1}{4}} n^{\frac{1}{4}}}{[\ln \frac{1}{1-\delta}]^{\frac{1}{4}} \sqrt{\sigma} T^{\frac{3}{4}}}\right)$. On the other hand, when $\sigma = 0$, both EF21-SGDM and our method set the momentum to 1. In this noiseless regime, EF21-SGDM uses a stepsize scaling as $\Omega(\delta)$, whereas our stepsize scales as $\Omega(1/\sqrt{T})$, which is smaller for large T .

Table I compares our proposed approach to EF21-SGDM. We can see both algorithms exhibit the same convergence rate, number of samples, and communication amounts when [22] runs for T iterations and our Algorithm 1 runs for T/u iterations. Indeed, EF21-SGDM has an important advantage over Algorithm 1: it does not introduce the auxiliary hyperparameter u , and therefore typically requires less tuning. Nevertheless, our method also offers a complementary benefit. In particular, when the noise level σ is extremely small, our bound reduces to $\tilde{\mathcal{O}}(1/\sqrt{T})$, which implies an iteration complexity of $\tilde{\Omega}(1/\epsilon^2)$ to reach ϵ -stationarity. This improves over the corresponding dependence reported for EF21-SGDM, which scales as $\Omega(1/(\delta\epsilon^2))$ reported in Corollary 1 in [22] especially for heavy compression (δ is small). Moreover, we want to stress again that our gradient normalization offers a unified mechanism capable of achieving state-of-the-art convergence rates across compressed, federated, and asynchronous distributed learning. In contrast, the approach in [22] is limited to communication compression. Evidently, our method provides broader applicability for distributed training tasks.

About the communication costs of MSC: It is straightforward to verify that our algorithm processes a total of $(T/u) \cdot u = T$ samples upon completion. Meanwhile, since each MSC operation takes u communication, our algorithm will execute a total of $(T/u) \cdot u = T$ rounds of communications upon completion, with each communication transmitting a *single* compressed vector (e.g., the top- K vector). Therefore, executing our modified algorithm for T/u rounds incurs the same amount of sample queries and communications as running other common compressed algorithms, such as QSGD [6] and EF21-SGDM [22], for T rounds. Notably, it does not incur more communication overhead than algorithms that do not utilize MSC. We list the detailed comparison in Table I.

TABLE I
A COMPREHENSIVE COMPARISON BETWEEN ALGORITHM 1 AND EF21-SGDM [22]

Algorithm	Iterations	Samples	Communicate with Top- K	Convergence rate
EF21-SGDM	T	T	TK	$\sqrt{\sigma}/(nT)^{1/4}$
Our algorithm	T/u	$T/u \times u = T$	$T/u \times (uK) = TK$	$\sqrt{\sigma}/(nT)^{1/4}$

TABLE II
A COMPREHENSIVE COMPARISON BETWEEN ALGORITHM 1 AND EXISTING ALGORITHMS, INCLUDING MEM-SGD [36], CSER [69], EF21-SGDM [22], AND NEOLITHIC [23]. “BG” INDICATES THE ASSUMPTION OF BOUNDED GRADIENT, AND “BGD” INDICATES THE ASSUMPTION OF BOUNDED GRADIENT DIFFERENCE. THE CONSTANT K IS FAR SMALLER THAN THE DIMENSION d OF THE VARIABLES.

Algorithm	Iter.	Samp.	Commun. using top- K	Conver. rate	Assump.
Mem-SGD	T	T	TK	$\sqrt{\sigma}/(nT)^{1/4}$	BG
CSER	T	T	TK	$\sqrt{\sigma}/(nT)^{1/4}$	BG
EF21-SGDM	T	T	TK	$\sqrt{\sigma}/(nT)^{1/4}$	None
NEOLITHIC	T/u	$T/u \times u = T$	$T/u \times (uK) = TK$	$\sqrt{\sigma}/(nT)^{1/4}$	BGD
Our algorithm	T/u	$T/u \times u = T$	$T/u \times (uK) = TK$	$\sqrt{\sigma}/(nT)^{1/4}$	None
No compression	T	T	Td	$\sqrt{\sigma}/(nT)^{1/4}$	None

IV. FEDAVG WITH GRADIENT NORMALIZATION

A. Federated Averaging

FedAvg [8], [9] has emerged as a prevalent algorithm for federated learning by performing multiple local stochastic gradient descent steps within each client before communication with the central server (see Algorithm 5 in Appendix A, online available). However, FedAvg is highly sensitive to heterogeneous data distributions across clients. Theoretically, its convergence relies on the restrictive bounded data heterogeneity assumption (2). When such an assumption is violated, the analysis breaks down. Empirically, FedAvg exhibits significantly deteriorated convergence with severe data heterogeneity.

B. The Gradient Normalization Algorithm for FedAvg

To mitigate the influence of heterogeneous data on FedAvg, we propose a novel normalized FedAvg algorithm outlined in Algorithm 2. In each iteration, each client first executes K local SGD steps and then updates its momentum. These momentum variables are transmitted to the central server, aggregated across clients, and normalized before using them to update the global model parameters. This new approach differs from vanilla FedAvg (Algorithm 5) in two key aspects:

- It incorporates momentum accumulation in the server for K local updates from clients.
- It employs gradient normalization rather than vanilla gradient averaging.

These modifications only marginally increase the storage for momentum vectors and the computational cost of calculating the norm. Critically, they introduce no additional communication per iteration beyond standard FedAvg.

C. Convergence Analysis

The following theorem establishes the convergence property of normalized FedAvg.

Algorithm 2. Normalized Federated Averaging.

Require: hyperparameters $\gamma, \eta > 0, 0 \leq \theta < 1$

Initialization: $\mathbf{w}^0, \mathbf{m}^0 = \mathbf{0}$

for iteration $t = 1, 2, \dots, T$

communicate the parameter to all clients

one client $i \in \{1, 2, \dots, n\}$ **in parallel** do

for $k = 0, 1, 2, \dots, K - 1$

step 1: Client i sets $\mathbf{y}^{t,0}(i) = \mathbf{w}^t$

Client i samples $\xi_i^{t,k} \sim \mathcal{D}_i$ to get

$\mathbf{g}^{t,k}(i) = \nabla f(\mathbf{y}^{t,k}(i); \xi_i^{t,k})$

Client i updates $\mathbf{y}^{t,k+1}(i) = \mathbf{y}^{t,k}(i) - \eta \mathbf{g}^{t,k}(i)$

step 2: Client i sends $\mathbf{w}^t - \mathbf{y}^{t,K}(i)$ to server

end on client

step 3: Server updates

$\mathbf{m}^t(i) = \theta \mathbf{m}^{t-1}(i) + \frac{(1-\theta)}{K\eta} (\mathbf{w}^t - \mathbf{y}^{t,K}(i))$ and

$\mathbf{w}^{t+1} = \mathbf{w}^t - \gamma \frac{\sum_{i=1}^n \mathbf{m}^t(i)}{\|\sum_{i=1}^n \mathbf{m}^t(i)\|}$ and sends it to all clients

end for

Theorem 2: Let $(\mathbf{w}^t)_{t \geq 0}$ be generated by Algorithm 2. Suppose Assumptions 1–2 hold. If $1 - \theta = \min \left\{ \frac{\sqrt{nK}}{\sigma\sqrt{T}}, 1 \right\}$, $\gamma = \frac{\sqrt{1-\theta}}{L\sqrt{T}}$, $\eta = \frac{\sigma}{4LK^{3/2}\sqrt{nT}}$, and $nTK \geq \sigma^2$, then it holds that

$$\begin{aligned} \frac{1}{T} \sum_{t=1}^T \mathbb{E} \|\nabla f(\mathbf{w}^t)\| &\leq \frac{\sqrt{\sigma}L(f(\mathbf{w}^0) - \min f) + 7\sqrt{\sigma}}{(nK)^{1/4}T^{1/4}} \\ &+ \frac{2\zeta_0\sigma + 4\|\nabla f(\mathbf{w}^0)\|\sigma + \sigma}{(nK)^{1/2}T^{1/2}} + \frac{(f(\mathbf{w}^0) - \min f)L + 7}{\sqrt{T}} \\ &+ \frac{2\zeta_0 + 4\|\nabla f(\mathbf{w}^0)\| + 1}{T}, \end{aligned}$$

where ζ_0 is a constant defined by (3).

Linear speedup without assuming bounded data heterogeneity: Theorem 2 implies that normalized FedAvg will converge at

a rate of $\mathcal{O}((\sqrt{\sigma})/((nK)^{1/4}T^{1/4}))$ when T grows sufficiently large. This suggests that to attain an ϵ -accurate solution, the number of iterations required by Algorithm 2 will decrease linearly with both the number of clients n and local updates K . Notably, this linear speedup property is achieved without assuming bounded data heterogeneity. On the other hand, we can see that under the same stochastic first-order oracle, the minimax lower bound implies $nKT = \Omega(\sigma^2/\epsilon^4)$ for achieving ϵ -stationary in $\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \|\nabla f(\mathbf{w}^t)\|$. Hence the dominant term in Theorem 2 is minimax-optimal with respect to the total stochastic gradients.

Comparison with FedAvg: In the homogeneous distributed learning, it has been extensively established (e.g. in [17], [25], [70], [71]) that FedAvg can achieve the linear speedup rate of $\mathcal{O}\left(\frac{\sqrt{\sigma}}{(nK)^{1/4}T^{1/4}}\right)$. However, the analysis of FedAvg under heterogeneity presents a considerably more intricate challenge. To derive guarantees of linear speedup, works such as [71] make the assumption of bounded gradients, while others like [17], [25] presume bounded data heterogeneity (as per condition 2). It is worth noting that these assumptions are often impractical in the context of non-convex stochastic learning. Compared to these references, our normalized FedAvg achieves a linear speedup rate without imposing any such restrictive assumptions.

Comparison with Scaffold: Note that the Scaffold algorithm achieves a convergence rate of $\mathcal{O}\left(\frac{\sqrt{\sigma}}{(nK)^{1/4}T^{1/4}} + \frac{1}{T^{1/2}}\right)$ [17], which is comparable to the rate of the normalized FedAvg algorithm in both dominant and high-order terms. While matching the rate, our method still provides a practical communication advantage: Scaffold requires communicating two vectors per round (the model update and the control correction term), whereas our normalized FedAvg communicates only *one* vector per round.

Comparison with FedCM/FedAvg-M: FedCM [72] or FedAvg-M [26] utilizes momentum to enhance the robustness to data heterogeneity. In particular, FedAvg-M provably removes the data heterogeneity assumption and achieves a convergence rate of $\mathcal{O}\left(\frac{1}{(nK)^{1/4}T^{1/4}} + \frac{1}{T^{1/2}}\right)$, which is the same as our rate achieved in Theorem 2. Even when we restrict gradient normalization to the federated setting, our method does enjoy several advantages. The FedAvg-M [26] implementation updates the client momentum using an estimate of the global stochastic gradient. In practice, this requires each worker to store an extra copy of the previous global model $\mathbf{w}^{t-1}$ (received from the server) in order to recover the global stochastic gradient, e.g. $\mathbf{g}^t = \frac{\mathbf{w}^{t-1} - \mathbf{w}^t}{\gamma}$, for updating. Therefore, FedAvg-M associates an additional per-worker storage with one model vector. On the other hand, one can observe that the convergence guarantee of FedAvg-M setting used by [26] is based on an implicit initialization condition. The Theorem 11 of [26] indeed assumes $\frac{1}{n} \sum_{i=1}^n \|\nabla f_i(\mathbf{w}^0)\|^2 < +\infty$. In contrast, our analysis only assumes that initialization heterogeneity is finite, namely $\zeta_0 = \frac{1}{n} \sum_{i=1}^n \|\nabla f_i(\mathbf{w}^0)\|^2 - \|\frac{1}{n} \nabla f(\mathbf{w}^0)\|^2 < +\infty$. We can see our condition is more relaxed. To cap it all, we call out attention to the fact of looking for a general gradient normalization based method as applied to the literature of federated optimization. The aforementioned process and regulatory framework can further

be applied to other communication-efficient distributed settings (e.g., compressed and asynchronous training), in which the analyses done in FedAvg-M do not have direct application.

V. AD-SGD WITH GRADIENT NORMALIZATION

A. Asynchronous Distributed SGD

The authors of [73] propose a pipeline-based method for the gradient descent (GD) algorithm. In this approach, the master node sequentially and cyclically interacts with each client node, gathering their latest stochastic gradients and relaying the updated iteration back to the client before proceeding to the next client. Mathematically, this scheme corresponds to a delayed GD algorithm. However, practical challenges arise when certain clients have difficulty meeting task deadlines. To address these issues, [55] proposed an alternative asynchronous parallel approach that eliminates the need for global synchronization.

The AD-SGD algorithm further considers the scenario where each client corresponds to a different dataset (see Algorithm 6 in Appendix A, online available). Notably, when client activation is implemented in a cyclic manner, AD-SGD follows a scheme similar to that presented in [55]. The present analysis of AD-SGD relies on the assumption of bounded data heterogeneity since all clients contribute biased stochastic gradients.

B. The Gradient Normalization Algorithm for AD-SGD

This subsection presents AD-SGD with gradient normalization, see Algorithm 3. In the algorithm, each client is equipped with a dedicated copy of the parameter. In each iteration, individual clients compute stochastic gradients using their the parameter copy. These calculated gradients are transmitted to the master node for consolidation. The master node aggregates the disparate gradients provided by the workers to update the global parameter with gradient normalization and momentum. After updating the global parameter in the master node, it is broadcast back to partial clients. Notably, the clients are able to continue their computations uninterrupted, making use of their current local parameter copies. Since the stochastic gradient $\mathbf{g}_i^t$ involves the copy of the parameter and may cause delay, we assume this delay to be $\tau_{i,t}$, i.e., $\mathbf{g}_i^t = \nabla f_i(\mathbf{w}^{t-\tau_{i,t}}; \xi_i^t)$. The new algorithm differs from the vanilla AD-SGD (Algorithm 6) in the following three aspects.

- The master client utilizes additional storage for the momentum, updating it after each iteration of reading and writing.
- The clients use extra storage for the copy of the parameter and use it for the stochastic gradient computation. The copy will be refreshed when the clients receive the updated parameter.
- The algorithm employs gradient normalization instead of using the vanilla gradient for parameter updates.

Similar to preceding algorithms, the modification to AD-SGD only incurs a marginal increase in the storage requirements and the computational cost. Importantly, this adaptation introduces no additional communication overhead.

Algorithm 3. Normalized Asynchronous Distributed SGD.

Require: hyperparameters $\gamma > 0, 0 \leq \theta < 1$
Initialization: $\mathbf{w}^0, \mathbf{m}^0 = \mathbf{0}$
for iteration $t = 1, 2, \dots, T$
 one client $i \in \{1, 2, \dots, n\}$ **asynchronously** do
 step 1: Sample $(\xi_i^t \sim \mathcal{D}_i)_{1 \leq i \leq n}$ with the **parameter copy** to calculate local stochastic gradient
 $\mathbf{g}^t(i) = \nabla f_i(\mathbf{w}^{t-\tau_{i,t}}; \xi_i^t)$
 and send it to the server
 end on client
 step 2: Server collects gradients to update
 $\mathbf{g}^t = \frac{\sum_{i=1}^n \mathbf{g}^t(i)}{n}$
 step 3: Server updates the momentum
 $\mathbf{m}^t = \theta \mathbf{m}^{t-1} + (1 - \theta) \mathbf{g}^t$
 step 4: Server updates the parameter
 $\mathbf{w}^{t+1} = \mathbf{w}^t - \gamma \frac{\mathbf{m}^t}{\|\mathbf{m}^t\|}$
 send the parameter to clients to update their **parameter copies**
end for

C. Convergence Analysis

In the analysis, we make the assumption of uniform boundedness on the delay, which is commonly used in asynchronous SGD [11], [12], [55], [56], [74].

Assumption 3: (Bounded Delay) The delay in Algorithm 3 is uniformly bounded, that is, there exists $\mathcal{T} > 0$ such that

$$\max_{1 \leq i \leq n, t \geq 0} \{\tau_{i,t}\} \leq \mathcal{T}.$$

Now, we are prepared to present the convergence of Algorithm 3.

Theorem 3: Let $(\mathbf{w}^t)_{t \geq 0}$ be generated by Algorithm 3, and Assumptions 1 and 3 hold. If $1 - \theta = \min\{\frac{\sqrt{n}}{\sigma\sqrt{T}}, 1\}$, $\gamma = \frac{\sqrt{1-\theta}}{L\sqrt{T}}$, we then get

$$\begin{aligned} \frac{1}{T} \sum_{t=1}^T \mathbb{E} \|\nabla f(\mathbf{w}^t)\| &\leq \frac{L\sqrt{\sigma}(f(\mathbf{w}^0) - \min f)}{n^{1/4}T^{1/4}} + \frac{9\sqrt{\sigma}}{2n^{1/4}T^{1/4}} \\ &+ \frac{2\zeta_0\sigma + 2\|\nabla f(\mathbf{w}^0)\|\sigma}{n^{1/2}T^{1/2}} + \frac{(f(\mathbf{w}^1) - \min f)L + 5 + 2T}{\sqrt{T}} \\ &+ \frac{2\zeta_0 + 2\|\nabla f(\mathbf{w}^0)\|}{T}, \end{aligned}$$

where ζ_0 is a constant defined by (3).

According to Theorem 3, the convergence rate of Algorithm 3 is $\mathcal{O}\left(\frac{\sqrt{\sigma}}{n^{1/4}T^{1/4}} + \frac{\gamma}{\sqrt{T}}\right)$ without relying on the bounded data heterogeneity assumption (2). To align with the asynchronous algorithm and establish a fair comparison, we set $\mathcal{K} := nT$ since AD-SGD employs a cyclic update. Algorithm 3 achieves a convergence rate of $\mathcal{O}\left(\frac{\sqrt{\sigma}}{\mathcal{K}^{1/4}} + \frac{\gamma}{\sqrt{\mathcal{K}}}\right) = \mathcal{O}\left(\frac{\sqrt{\sigma}}{\mathcal{K}^{1/4}}\right)$. This interesting result indicates that the delay has a minimal impact on the overall speed compared to other factors, consistent with the findings in [12]. Note that asynchrony does not provide additional oracle information beyond stochastic gradients; it only changes the arrival of gradient information. Therefore,

the minimax lower bound under the stochastic first-order oracle also applies to the asynchronous regime: achieving ϵ -stationarity requires $\Omega(\sigma^2/\epsilon^4)$ in general, equivalently the dominant dependence cannot be better than $\Omega(\sqrt{\sigma}/(nT)^{1/4})$ (the total number of stochastic gradient evaluations is nT). Hence the rate in Theorem 3 is minimax-optimal in the dominant term with respect to the total stochastic gradients, and the delay affects only subordinate terms.

Under the scenario of heterogeneous data, [12] establish the convergence of asynchronous SGD as $\mathcal{O}\left(\frac{\sqrt{\sigma} + \sqrt{\zeta}}{\mathcal{K}^{1/4}}\right)$, where the delay is within the subordinated term of the convergence rate. Notably, our algorithm performs significantly better when severe data heterogeneity exists (ζ being large) as the convergence of Algorithm 3 is not contingent on ζ . When considering the subordination rate, our rate does not outperform that of [12], as they employ the average delay. Nonetheless, as previously asserted, the delay is encompassed within the subordinated component of the convergence rate. Thus, our method remains superior.

In contrast to [60], our study delves into the asynchronous setting with a particular emphasis on the heterogeneous data scenario. In our scenario, the data distribution $\mathcal{D}_i$ on node i may vary, contrasting with the assumption made in [60] where $\mathcal{D}_i$ are presumed to be uniform (refer to (1) in [60]). Consequently, the straightforward fulfillment of the bounded heterogeneous data assumption ((2) in our paper) ensues. The key innovation of our asynchronous findings lies in showcasing that the bounded data heterogeneity assumption can be eliminated through the utilization of gradient normalization and momentum.

VI. NUMERICAL EXPERIMENTS

This section examines the performance of gradient normalization when applied to CD-SGD, FedAvg, and AD-SGD. We first present comparative experiments on binary logistic regression with non-convex regularized terms. And then, we provided experimental results of multi-class logistic regression on MLP and ResNet model in the Appendix B, online available. It is worth noting that in the binary logistic regression experiments, to more accurately reflect the theoretical results, our experimental plots depict gradient norm vs. iteration, where the gradient norm refers to $\frac{1}{T} \sum_{t=1}^T \|\nabla f(\mathbf{w}^t)\|$. In contrast, for the MLP and ResNet experiments, we measure test accuracy or communication cost vs. iteration. The implementation code for our experiments is publicly available at: [https://github.com/wbhbhb/Gradient-Normalization-Distributed.git].

A. Basic Setup

The considered non-convex stochastic problem is as follows

$$\min_{\mathbf{w} \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{w}) + \rho r(\mathbf{w}), \quad (10)$$

where $f_i(\mathbf{w}) = \frac{1}{m} \sum_{\ell=1}^m \ln(1 + \exp(-y_{i,\ell} x_{i,\ell}^T \mathbf{w}))$, $r(\mathbf{w}) = \sum_{j=1}^d \frac{\mathbf{w}(j)^2}{1 + \mathbf{w}(j)^2}$. In problem (10), $\mathbf{w}(j)$ is the j -th element in

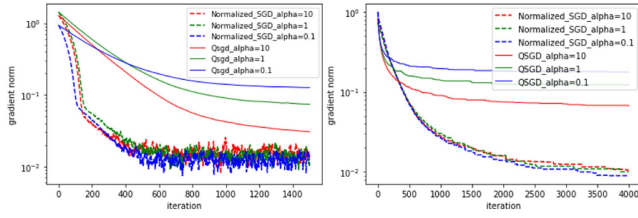


Fig. 1. Performance comparison between QSGD and normalized SGD with multi-step compression. Left: synthesized dataset. Right: MNIST dataset.

$\mathbf{w}$, m is the size of local dataset, $(x_{i,\ell}, y_{i,\ell})_{1 \leq \ell \leq m}$ are the local dataset kept by client i in which $x_{i,\ell} \in \mathbb{R}^d$ is the feature vector and $y_{i,\ell} \in \{+1, -1\}$ is the associated label. Problem (10) is non-convex due to the regularizer $r(\mathbf{w})$. The regularization optimization employed in our numerical experiments have been extensively researched in previous works. Please refer to [Eq. (13), [27]], [Section 6, [75]], and [Section 4.3, [76]].

Dataset with heterogeneous distributions: We consider two types of datasets in our experiments: a synthesized dataset and the real-world MNIST dataset. In the synthesized dataset, we generate data following C distinct distributions where $C = 40$. To produce data samples in the c -th distribution, we first generate $\mathbf{w}_c^* = \mathbf{w}^* + v_c$ where $\mathbf{w}^* \sim \mathcal{N}(0, I_d)$ is a randomly generated vector and $v_c \sim \mathcal{N}(0, \sigma_h^2 I_d)$ controls the similarity between each local solution. Generally speaking, a large σ_h^2 results in local solutions $\mathbf{w}_c^*$ that are very different from each other. In distribution c , we generate $x_{c,\ell} \sim \mathcal{N}(0, 1)$ and $z_{c,\ell} \sim \mathcal{U}(0, 1)$. If $z_{c,\ell} \leq 1 + \exp(-y_{c,\ell} x_{c,\ell}^T \mathbf{w}_c^*)$, we set $y_{c,\ell} = 1$; otherwise $y_{c,\ell} = -1$. The MNIST dataset has $C = 10$ classes, and each class can be regarded as a data distribution. We designate images with labels 0, 1, 2, 3, 4 as positive samples while the others are negative samples.

Sample data from different distributions: In all experiments, each client samples data from C distinct distributions. The Dirichlet distribution α values control data heterogeneity between clients [77], [78]. Smaller α values make it more likely that clients hold examples from only one class, causing significant data heterogeneity. Larger α values make it more likely that clients sample data evenly from all distributions, significantly reducing data heterogeneity. Our experiment will examine the performance of gradient normalization across different α values.

B. Numerics for CD-SGD With Gradient Normalization

Comparison with QSGD: QSGD [6] stands out as a widely adopted algorithm leveraging communication compression, a technique recognized for its susceptibility to the impact of data heterogeneity. In Fig. 1, we present a comparative analysis of distributed normalized SGD with MSC against QSGD across 100 clients. In the case of synthesized data, each algorithm employs a top-2 compressor alongside a constant learning rate. Meanwhile, for MNIST, each algorithm opts for a top-50 compressor and employs a decaying learning rate, halving its value after every 2000 iterations. Our evaluation of these algorithms encompasses three distinct levels of data heterogeneity: slight ($\alpha = 10$), medium ($\alpha = 1$), and severe ($\alpha = 0.1$).

The left plot depicts experiments with synthesized data. It is observed that the performance of QSGD varies drastically under different levels of data heterogeneity. With high data heterogeneity ($\alpha = 0.1$), QSGD converges to a solution with significantly worse accuracy. In contrast, normalized SGD maintains consistent performance regardless of varying data heterogeneities, demonstrating robustness to heterogeneous data distributions, aligning with our Theorem 1. Moreover, normalized SGD is observed to converge much faster than QSGD in the presence of data heterogeneity. The right plot depicts experiments with MNIST data. Similarly, it shows that normalized SGD is more robust to data heterogeneity and converges to a much more accurate solution than QSGD.

Comparison with error feedback: Fig. 2 compares our proposed normalized SGD with two communication-compressed methods that use error feedback Mem-SGD [36] and EF21-SGD [21]. These methods are recognized for mitigating the impact of data heterogeneity. The experimental configurations align with those presented in Fig. 1.

The left and middle plots depict performance comparisons on synthesized data. Remarkably, all algorithms converge asymptotically to consistent accuracy levels, irrespective of variations in data heterogeneity. This underscores the robustness of our proposed algorithm to data heterogeneity, on par with Mem-SGD and EF21-SGD. Additionally, our proposed algorithm exhibits similar convergence performance to Mem-SGD and EF21-SGD when utilizing constant learning rates. The right plot portrays severe data heterogeneity ($\alpha = 0.1$) on the MNIST dataset. Here, it is evident that normalized SGD with multi-step compression outperforms the convergence of the other algorithms, when employing a decaying learning rate, as the iteration count increases.

C. Numerics for FedAvg With Gradient Normalization

This subsection seeks to assess the efficacy of gradient normalization in enhancing FedAvg's performance in the presence of data heterogeneity. Fig. 3 presents the performance comparison of these two algorithms across 100 clients. For synthesized data, each algorithm employs 10 local steps per iteration. For MNIST, both algorithms adopt 30 local steps per iteration. A constant learning rate is used in both experiments. Furthermore, the momentum parameters in FedAvg with gradient normalization are set to $\theta = 0.2$ for synthesized data and $\theta = 0.9$ for MNIST.

The left plot illustrates performance comparisons on synthesized data, revealing a pronounced impact of data heterogeneity on the convergence performance of FedAvg. As data heterogeneity increases, FedAvg tends to converge toward a less accurate solution. In contrast, regardless of the degree of data heterogeneity, FedAvg with gradient normalization consistently converges to an accurate solution, emphasizing its robustness in the face of heterogeneous data distributions. The right plot demonstrates that, with a decaying learning rate, gradient normalization consistently and significantly accelerates FedAvg across different levels of data heterogeneity (i.e., $\alpha = 2.5$, $\alpha = 0.5$, or $\alpha = 0.1$).

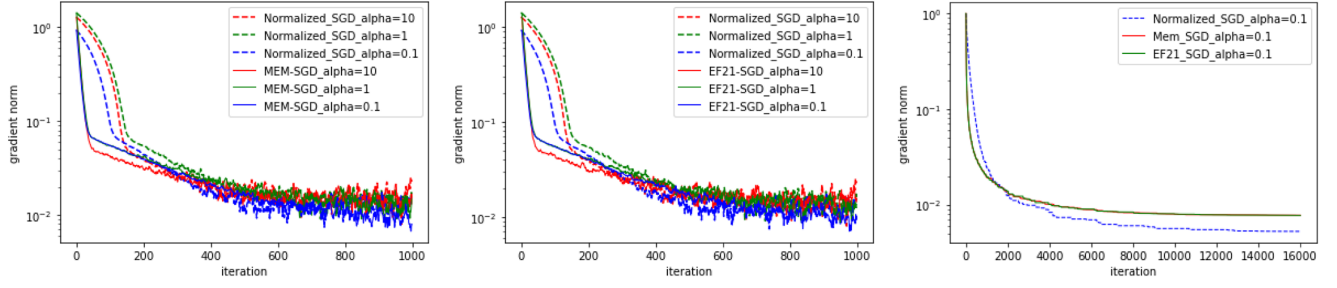


Fig. 2. Performance comparison between Mem-SGD, EF21-SGD and normalized SGD with multi-step compression. Left and Middle: synthesized dataset. Right: MNIST dataset.

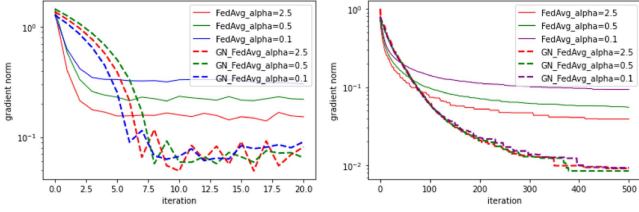


Fig. 3. Performance comparison between vanilla FedAvg and gradient-normalized (GN) FedAvg. Left: synthesized dataset. Right: MNIST dataset.

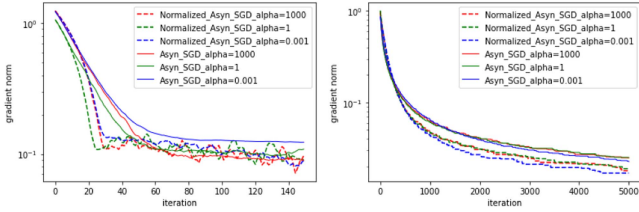


Fig. 4. Performance comparison between vanilla AD-SGD and gradient-normalized AD-SGD. Left: synthesized dataset. Right: MNIST dataset.

D. Numerics for AD-SGD With Gradient Normalization

This subsection aims to evaluate the impact of gradient normalization on enhancing the performance of AD-SGD in the presence of data heterogeneity. Fig. 4 presents a comparative analysis of these two algorithms across 30 clients. In the case of synthesized data, each algorithm utilizes a constant learning rate, while for MNIST, both algorithms adopt a decaying learning rate. Additionally, in FedAvg with gradient normalization, the momentum parameters are set to $\theta = 0.2$ for synthesized data and $\theta = 0.9$ for MNIST.

The left plot depicts performance comparisons on synthesized data, revealing that as data heterogeneity increases, AD-SGD tends to converge toward a less accurate solution. Conversely, irrespective of the degree of data heterogeneity, AD-SGD with gradient normalization consistently converges to an accurate solution, highlighting its robustness in the face of heterogeneous data distributions. Moreover, gradient normalization induces a slightly faster convergence for AD-SGD compared to vanilla AD-SGD. The right plot demonstrates that, with a decaying learning rate, gradient normalization consistently improves the convergence of AD-SGD.

VII. CONCLUSION

In this paper, we explore the advantages of gradient normalization in various heterogeneous training scenarios such as compressed distributed training, federated training, and asynchronous training. Our main contribution is proving that the assumption of bounded data heterogeneity is unnecessary when gradient normalization is employed. This result indicates that our proposed algorithm can achieve better convergence even when the data is highly diverse. To validate our theoretical findings, we provide numerical results that demonstrate the effectiveness and efficiency of gradient normalization.

REFERENCES

- [1] H. Robbins and S. Monro, "A stochastic approximation method," *Ann. Math. Statist.*, vol. 22, pp. 400–407, 1951.
- [2] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *Proc. COMPSTAT: 19th Int. Conf. Comput. Statistics Paris France*, 2010, pp. 177–186.
- [3] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Rev.*, vol. 60, no. 2, pp. 223–311, 2018.
- [4] M. Li, T. Zhang, Y. Chen, and A. J. Smola, "Efficient mini-batch training for stochastic optimization," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 661–670.
- [5] F. Seide, H. Fu, J. Droppo, G. Li, and D. Yu, "1-bit stochastic gradient descent and its application to data-parallel distributed training of speech DNNs," in *Proc. 15th Annu. Conf. Int. Speech Commun. Assoc.*, 2014, pp. 1058–1062.
- [6] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017.
- [7] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "SignSGD: Compressed optimisation for non-convex problems," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 560–569.
- [8] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," 2016, *arXiv:1610.05492*.
- [9] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statistics*, 2017, pp. 1273–1282.
- [10] T. Chen, G. Giannakis, T. Sun, and W. Yin, "LAG: Lazily aggregated gradient for communication-efficient distributed learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [11] X. Lian, Y. Huang, Y. Li, and J. Liu, "Asynchronous parallel stochastic gradient for nonconvex optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.
- [12] A. Koloskova, S. U. Stich, and M. Jaggi, "Sharper convergence guarantees for asynchronous SGD for distributed and federated learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 17202–17215.
- [13] K. Mishchenko, F. Bach, M. Even, and B. E. Woodworth, "Asynchronous SGD beats minibatch SGD under arbitrary delays," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 420–433.
- [14] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," 2018, *arXiv:1806.00582*.

- [15] Y. Chen, K. Yuan, Y. Zhang, P. Pan, Y. Xu, and W. Yin, "Accelerating gossip SGD with periodic global averaging," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 1791–1802.
- [16] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-IID data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3400–3413, Sep. 2020.
- [17] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5132–5143.
- [18] A. Koloskova, N. Loizou, S. Boreiri, M. Jaggi, and S. Stich, "A unified theory of decentralized SGD with changing topology and local updates," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5381–5393.
- [19] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2013, pp. 1310–1318.
- [20] S. P. Karimireddy, Q. Rebjock, S. Stich, and M. Jaggi, "Error feedback fixes SignSGD and other gradient compression schemes," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 3252–3261.
- [21] P. Richtárik, I. Sokolov, and I. Fatkhullin, "Ef21: A new, simpler, theoretically better, and practically faster error feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 4384–4396.
- [22] I. Fatkhullin, A. Tyurin, and P. Richtárik, "Momentum provably improves error feedback!," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 36, pp. 76444–76495.
- [23] X. Huang, Y. Chen, W. Yin, and K. Yuan, "Lower bounds and nearly optimal algorithms in distributed learning with communication compression," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 18955–18969.
- [24] Y. He, X. Huang, Y. Chen, W. Yin, and K. Yuan, "Lower bounds and accelerated algorithms in distributed stochastic optimization with communication compression," 2023, *arXiv:2305.07612*.
- [25] H. Yang, M. Fang, and J. Liu, "Achieving linear speedup with partial worker participation in non-IID federated learning," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [26] Z. Cheng, X. Huang, and K. Yuan, "Momentum benefits non-IID federated learning simply and provably," 2023, *arXiv:2306.16504*.
- [27] X. Zhang, M. Hong, S. Dhople, W. Yin, and Y. Liu, "FedPD: A federated learning framework with adaptivity to non-IID data," *IEEE Trans. Signal Process.*, vol. 69, pp. 6055–6070, 2021.
- [28] D. A. E. Acar, Y. Zhao, R. M. Navarro, M. Mattina, P. N. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization," 2021, *arXiv:2111.04263*.
- [29] P. Jiang and G. Agrawal, "A linear speedup analysis of distributed deep learning with sparse and quantized communication," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [30] H. Tang, S. Gan, C. Zhang, T. Zhang, and J. Liu, "Communication compression for decentralized training," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [31] H. Zhang, J. Li, K. Kara, D. Alistarh, J. Liu, and C. Zhang, "ZipML: Training linear models with end-to-end low precision, and a little bit of deep learning," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 4035–4043.
- [32] J. Wu, W. Huang, J. Huang, and T. Zhang, "Error compensated quantized SGD and its applications to large-scale distributed optimization," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5325–5333.
- [33] T. Sun, Q. Wang, D. Li, and B. Wang, "Momentum ensures convergence of SignSGD under weaker assumptions," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 33077–33099.
- [34] J. Wangni, J. Wang, J. Liu, and T. Zhang, "Gradient sparsification for communication-efficient distributed optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [35] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2017, pp. 440–445.
- [36] S. U. Stich, J.-B. Cordonnier, and M. Jaggi, "Sparsified SGD with memory," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [37] L. Condat, K. Yi, and P. Richtárik, "EF-BV: A unified theory of error feedback and variance reduction mechanisms for biased and unbiased compression in distributed optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 17501–17514.
- [38] R. McDonald, M. Mohri, N. Silberman, D. Walker, and G. Mann, "Efficient large-scale distributed training of conditional maximum entropy models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 22, 2009.
- [39] M. Zinkevich, M. Weimer, L. Li, and A. Smola, "Parallelized stochastic gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 23, 2010.
- [40] F. Zhou and G. Cong, "On the convergence properties of a k-step averaging stochastic gradient descent algorithm for nonconvex optimization," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 3219–3227.
- [41] M. R. Glasgow, H. Yuan, and T. Ma, "Sharp bounds for federated averaging (local SGD) and continuous perspective," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2022, pp. 9050–9090.
- [42] S. U. Stich, "Local SGD converges fast and communicates little," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [43] A. Khaled, K. Mishchenko, and P. Richtárik, "Tighter theory for local SGD on identical and heterogeneous data," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 4519–4529.
- [44] B. Woodworth et al., "Is local SGD better than minibatch SGD?," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 10334–10343.
- [45] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [46] F. Haddadpour, M. M. Kamani, M. Mahdavi, and V. Cadambe, "Trading redundancy for communication: Speeding up distributed SGD for non-convex optimization," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2545–2554.
- [47] H. Yu and R. Jin, "On the computation and communication complexity of parallel SGD with dynamic batch sizes for stochastic non-convex optimization," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7174–7183.
- [48] H. Yu, R. Jin, and S. Yang, "On the linear speedup analysis of communication efficient momentum SGD for distributed non-convex optimization," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7184–7193.
- [49] T. Sun, D. Li, and B. Wang, "Decentralized federated averaging," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4289–4301, Apr. 2023.
- [50] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazaeni, "Federated learning with matched averaging," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [51] S. J. Reddi et al., "Adaptive federated optimization," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [52] H. Yuan and T. Ma, "Federated accelerated stochastic gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 5332–5344.
- [53] K. Huang, X. Li, and S. Pu, "Distributed stochastic optimization under a general variance condition," 2023, *arXiv:2301.12677*.
- [54] J. Tsitsiklis, D. Bertsekas, and M. Athans, "Distributed asynchronous deterministic and stochastic gradient optimization algorithms," *IEEE Trans. Autom. Control*, vol. 31, no. 9, pp. 803–812, Sep. 1986.
- [55] A. Agarwal and J. C. Duchi, "Distributed delayed stochastic optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 24, 2011.
- [56] B. Recht, C. Re, S. Wright, and F. Niu, "Hogwild!: A lock-free approach to parallelizing stochastic gradient descent," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 24, 2011.
- [57] S. Sra, A. W. Yu, M. Li, and A. Smola, "AdaDelay: Delay adaptive distributed stochastic optimization," in *Proc. Artif. Intell. Statistics*, 2016, pp. 957–965.
- [58] Z. Zhou et al., "Distributed asynchronous optimization with unbounded delays: How slow can you go?," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5970–5979.
- [59] T. Sun, R. Hannah, and W. Yin, "Asynchronous coordinate descent under more realistic assumptions," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [60] A. Tyurin and P. Richtárik, "Optimal time complexities of parallel stochastic optimization methods under a fixed computation model," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024.
- [61] E. Hazan, K. Levy, and S. Shalev-Shwartz, "Beyond convexity: Stochastic quasi-convex optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.
- [62] K. Y. Levy, "The power of normalization: Faster evasion of saddle points," 2016, *arXiv:1611.04831*.
- [63] A. C. Wilson, L. Mackey, and A. Wibisono, "Accelerating rescaled gradient descent: Fast optimization of smooth functions," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [64] A. Cutkosky and H. Mehta, "Momentum improves normalized SGD," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 2260–2268.
- [65] T. Sun, Q. Wang, D. Li, and B. Wang, "Nonconvex Markov chain gradient descent under weak smoothness," Oct. 2023. [Online]. Available: https://www.researchgate.net/publication/375082810_Nonconvex_Markov_Chain_Gradient_Descent_under_Weak_Smoothness
- [66] Y. You, I. Gitman, and B. Ginsburg, "Large batch training of convolutional networks," 2017, *arXiv:1708.03888*.
- [67] Y. You et al., "Large batch optimization for deep learning: Training bert in 76 minutes," 2019, *arXiv:1904.00962*.

- [68] Y. Arjevani, Y. Carmon, J. C. Duchi, D. J. Foster, N. Srebro, and B. Woodworth, "Lower bounds for non-convex stochastic optimization," *Math. Program.*, vol. 199, no. 1–2, pp. 165–214, 2023.
- [69] C. Xie, S. Zheng, S. Koyejo, I. Gupta, M. Li, and H. Lin, "CSER: Communication-efficient SGD with error reset," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 12593–12603.
- [70] S. U. Stich, "Local SGD converges fast and communicates little," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [71] H. Yu, S. Yang, and S. Zhu, "Parallel restarted SGD with faster convergence and less communication: Demystifying why model averaging works for deep learning," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, pp. 5693–5700.
- [72] J. Xu, S. Wang, L. Wang, and A.C.-C. Yao, "FedCM: Federated learning with client-level momentum," 2021, *arXiv:2106.10874*.
- [73] M. Zinkevich, J. Langford, and A. Smola, "Slow learners are fast," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 22, 2009.
- [74] T. Sun, Y. Sun, D. Li, and Q. Liao, "General proximal incremental aggregated gradient algorithms: Better and novel results under general scheme," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [75] L. Luo, H. Ye, Z. Huang, and T. Zhang, "Stochastic recursive gradient descent ascent for stochastic nonconvex-strongly-concave minimax problems," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2022, pp. 20566–20577.
- [76] D. Hajinezhad and M. Hong, "Perturbed proximal primal–dual algorithm for nonconvex nonsmooth optimization," *Math. Program.*, vol. 176, no. 1, pp. 207–245, 2019.
- [77] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," 2019, *arXiv:1909.06335*.
- [78] T. Lin, S. P. Karimireddy, S. U. Stich, and M. Jaggi, "Quasi-global momentum: Accelerating decentralized deep learning on heterogeneous data," in *Proc. Int. Conf. Mach. Learn.*, vol. 139, 2021.



Tao Sun received the PhD degree from the National University of Defense Technology, in 2018. He is currently an associate professor with the College of Computer Science and Technology, National University of Defense Technology. His research interests include optimization for machine learning, reinforcement learning, distributed systems, and neural networks.



Baihao Wu is currently working in data science with the Academy for Advanced Interdisciplinary Studies, Peking University, Beijing, China. His research interests include theoretical and algorithmic foundations of optimization, deep learning, multi-precision training, and data science.



Xinwang Liu (Senior Member, IEEE) received the PhD degree from the National University of Defense Technology (NUDT), China. He is currently a full professor with the School of Computer, NUDT. He has authored or coauthored more than 60 peer-reviewed papers, including those in highly regarded journals and conferences such as *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Image Processing*, *IEEE Transactions on Neural Networks and Learning Systems*, *IEEE Transactions on Multimedia*, *IEEE Transactions on Information Forensics and Security*, *NeurIPS*, *ICCV*, *CVPR*, *AAAI*, and *IJCAI*. His research interests include kernel learning and unsupervised feature learning.



Kun Yuan (Member, IEEE) received the PhD degree in electrical and computer engineering from the University of California, Los Angeles, CA, USA, in 2019. From 2018 to 2019, he was a visiting researcher with École Polytechnique Fédérale de Lausanne, Switzerland, and a research intern with Microsoft Research Redmond, WA, USA. He was a staff algorithm engineer with the Decision Intelligence Laboratory, Alibaba (US) Group. He is currently an assistant professor with the Center for Machine Learning Research, Peking University, Beijing, China. His research interests include optimization theory, distributed algorithms, federated learning, and efficient systems for large-scale machine learning. He was the recipient of IEEE Signal Processing Society Young Author Best Paper Award in 2017 and IEEE CloudCom Distinguished Paper Award in 2025.